TASK BOARD
PROJECT REPORT

Arjun Shende (2024CS10252)
Rami Jay Ketanbhai (2024CS10510)

Table of Contents

1 Introduction

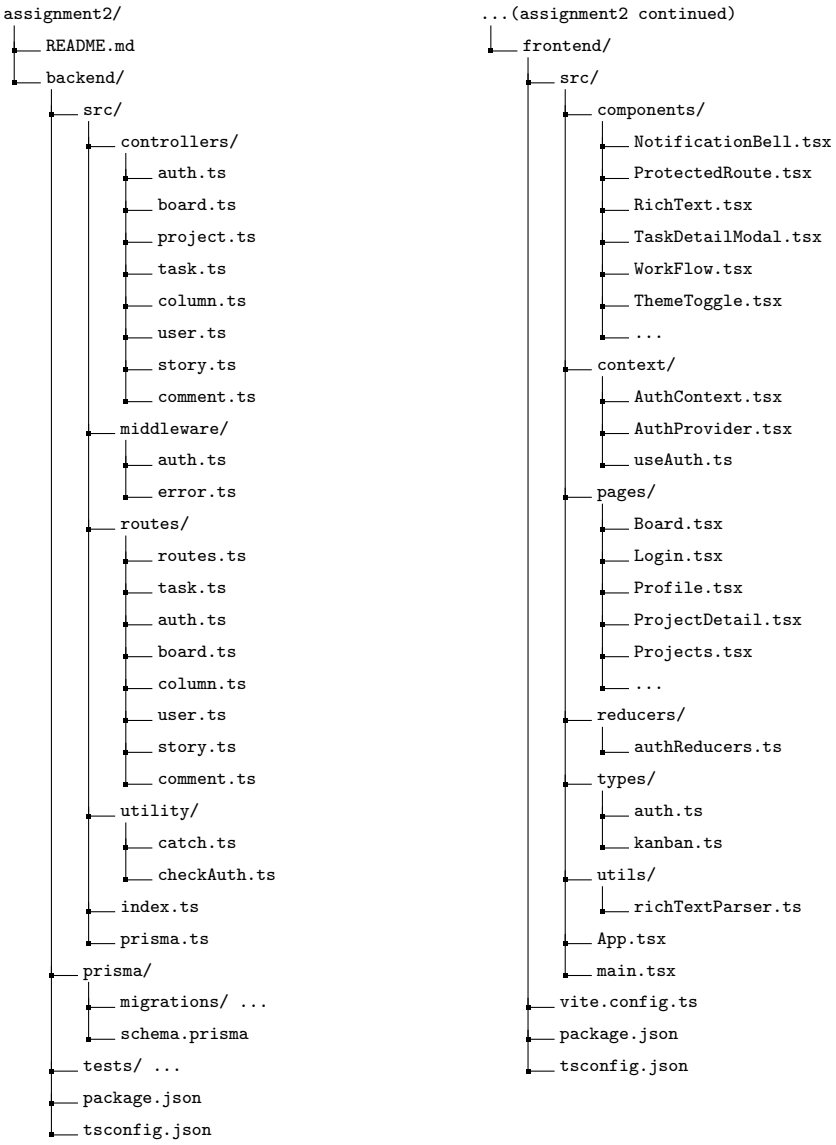
The Task Board project is a comprehensive, full-stack project management and issue-tracking application developed for COP290 - Design Practices Assignment 2. Inspired by industry-standard tools like Jira, the platform is designed to help agile teams plan, track, and manage their collaborative workflows efficiently.Built entirely with strict TypeScript, the application relies on a modern web architecture. The frontend is powered by React and Vite , utilizing Context and Reducer for state management. The backend consists of an Express.js REST API , secured by JSON Web Tokens (JWT) for session management. Data is persisted using a relational database (Postgres) alongside the Prisma ORM, ensuring end-to-end type safety and strict data integrity.

2 Repository Details

Both team members have actively committed code from their respective personal GitHub accounts. Here is the repository:

- **GitHub Repository Link:** CLICK HERE
- **Team Members:** Arjun Shende (ARK-LEGION) (2024CS10252) & Rami Jay Ketanbhai (Jayrami1) (2024CS10510)

3 Directory Structure



4 Design Decisions

To build this project management application, the following modern tech stack was utilized to ensure type safety, scalability, and a high-quality user experience:

- Typescript
- React + Vite
- Express.js

- PostgreSQL
- Prisma ORM
- JWT (Jason Web Tokens)
- CSS Modules
- Native HTML drag and drop API

4.1 Database Design and Schema architecture

The database is built on PostgreSQL and managed via the Prisma ORM. The schema is designed to ensure strict data integrity, support hierarchical issue tracking, and maintain a rigorous audit trail as mandated by the project requirements.

1. Role-Based Access Control (RBAC) & User Management: User permissions are highly granular and scoped at the project level.

- To achieve this, a Many-to-Many (M:N) relationship was implemented between the `User` and `Project` models through a join table named `Project_Group`.
- This join table stores the user's `role` (using the `Role` enum: `PROJECT_ADMIN`, `PROJECT_MEMBER`, `PROJECT_VIEWER`) specific to that exact project, allowing a single user to be an Admin in one project and a Viewer in another. A composite unique constraint (`@@unique([userId, projectId])`) ensures duplicate assignments are blocked.
- Global administrators are handled via a simple `isGLOBAL_ADMIN` boolean on the `User` model, bypassing project-specific checks.

2. Kanban Board & Column Workflow: The board structure is designed to be fully customizable per project.

- A `Project` can have multiple `Boards`, and each `Board` has multiple `Columns`.
- Each `Column` features an `order` integer to maintain its drag-and-drop position and an optional `wipLimit` to enforce "Work In Progress" constraints.
- Crucially, because columns can have custom names, each column is strictly mapped to an internal system state via the `cStatus` enum (`TO_DO`, `IN_PROGRESS`, `REVIEW`, `DONE`). This ensures the backend logic always understands the true state of a task, regardless of what the user names the column.

3. Task Hierarchy & Denormalization: The `Task` model is the core of the application, utilizing self-referencing relationships to support hierarchical structures (Stories, Tasks, and Bugs).

- **Hierarchy:** The `parentId` and `subtasks` fields create a one-to-many self-relation. This allows a 'Story' to act as a parent container for multiple 'Tasks' or 'Bugs'.
- **Dual-User Assignment:** Tasks maintain two separate relationships with the `User` model: the `assignee` (who is working on it) and the `reporter` (who created it).
- **Performance Optimization (Denormalization):** Originally, a task's project could only be determined by traversing `Task -> Column -> Board -> Project`. To optimize database read speeds and simplify authorization checks, a direct `projectId` foreign key was added to the `Task` model.

4. Audit Trail & Traceability: To fulfill the mandatory audit requirements, an `AuditLog` model is explicitly defined.

- It captures the exact `timestamp`, `action`, `oldValue`, and `newValue` of any modification.
- It maintains relations to both the `Task` (issue) being modified and the `User` (actor) who made the change.
- **Data Integrity:** The relation to the `Task` uses `onDelete: Cascade` (so logs are cleaned up if a task is deleted), but the relation to the `User` uses `onDelete: SetNull`. This is a critical design choice: if a user leaves the company and their account is deleted, the historical audit logs for the tasks they modified will remain intact, simply showing a `null` user, preserving the project's history.

4.2 Operational Design Decisions

- Conventionally, the users are of 4 types - GLOBAL ADMIN, PROJECT ADMIN, PROJECT MEMBER, PROJECT VIEWER (in decreasing order of hierarchy). Whenever a new account is created the user is not allocated any of these statuses by default. We can tweak someone's status to GLOBAL ADMIN as true by directly modifying the database (using commands like `npx prisma studio`). Note that by design this is the only way someone can become the GLOBAL ADMIN.

- Once we have a global admin in the database, he can assign statuses of ADMIN, VIEWERS, MEMBERS for a specific project to other users in the database.

- **Multiple Global Admins can be made via the database and they can still be assigned roles in projects while maintaining their admin privileges. A global admin has the privileges to modify anything anywhere across all projects.**

- For a project, the PROJECT ADMIN holds the power to assign any user to MEMBER/ADMIN/VIEWER roles. **THEY CANNOT CHANGE THEIR OWN ROLE.** Only the global and the project admins have access to the list of all the members in the database.

- The main application consists of multiple projects. Within a single project we can have multiple boards and stories. Each board has multiple columns with customizable workflows.

- Creation of new projects can only be done by GLOBAL ADMIN. Creation of Boards within a project can be done solely by the PROJECT ADMIN. Only PROJECT MEMBERS and above can create stories and tasks/bugs.

- A PROJECT VIEWER can only view the project and the contained tasks and stories but cannot modify them. Also they can do so only for the project they have been assigned. **ONLY ADMINS CAN DELETE A STORY**. Upon deletion of a story, all its associated tasks become as standalone.

- Only Admins can link a task to a story if the task already on the board. If a PROJECT MEMBER creates a task they may link ot to a story only at task creation time.

- Multiple tasks within columns across various boards can be linked to **stories**. Each story shows its children tasks along with the status derived from the status of its children tasks.

- **NO UPDATES CAN BE MADE TO AN ARCHIVED TASK, EVEN BY THE GLOBAL ADMIN**

- A single board has 4 default columns (TO DO, IN PROGRESS, REVIEW, DONE). However the columns and associated workflows are customizable and the user can define the allowed transitions (ADMINS ONLY). Only ADMINS can create/rename/reorder/delete/set WIP of the columns. Click on the column name to rename.

- New columns can be created by the ADMINS and by custom workflow they (only them) can set the name and mapped STATUS (TO DO/IN PROGRESS/REVIEW/DONE) and also define custom transitions between and across these columns.

- Tasks/Bugs reside within columns. Anyone with a status of PROJECT MEMBER and above can create tasks. Only ADMINS can set the name/due date/priority/assignee/description of a particular task. If a task is created by a project member he can only set the name of the task and that too at the time of creation. ADMINS can create and edit tasks completely....i.e. set the name/assignee/due date/priority/description of that task.

- If a task is assigned to an assignee then no other PROJECT MEMBER can edit anything in the task or even change its status. The assignee in this case can only drag and drop the task or change its status and not alter any other detail of the task. However if a task is unassigned, any MEMBER can drag and drop it and thereby change its status (not anything else). The status of a task is defined by the column in which it resides.

- If a task changes its status and there are more than one corresponding columns mapping to that status then we choose the first column where WIP limit won't be exceeded. Transitions across columns are defined between statuses. The task details STATUS dropdown allows the user to drop the task from the current column to any available column of the desired status. If there are no columns mapping to that status, throw an error.

- Anyone above or equivalent to a PROJECT MEMBER can comment on any task in the project and has the freedom to tag any other user. MEMBERS can only edit and delete their own comments. ADMINS can delete the comments of other users but not edit them.

- When the task details of an assigned task are updated the notification is broadcast to the concerned assignee. But when the task details of an unassigned task are updated there is no broadcast of notifications.

- Only the global admin can archive/unarchive projects and view the list of archived projects. Only GLOBAL and PROJECT admins can change board workflows. Workflows are unique per board and are customizable.

5 Features

Interactive Kanban Workspaces

- **Native Drag-and-Drop:** Users can seamlessly transition tasks across workflow columns using the native HTML5 Drag and Drop API, providing immediate visual feedback without relying on external UI libraries.

- **Smart WIP Enforcement:** The board actively monitors Work-In-Progress (WIP) limits. If a user attempts to drop a task into a column that has reached its maximum capacity, the interface proactively blocks the invalid status transition.

Rich Collaboration Tools

- **Markdown Support:** Task descriptions and comments support custom Rich Text formatting, allowing team members to communicate clearly with structured text.

- **User Mentions:** Team members can tag colleagues in comment threads using the `@username` notation, which automatically links to their profile and triggers a direct alert.

Real-Time Notification Center

- **In-App Alerts:** A persistent notification bell keeps users informed of critical updates, such as being assigned a new task, task status changes, or being mentioned in a conversation.

- **Inbox Management:** Users can view their historical notification feed, click through directly to the referenced task, and mark alerts as read to clear their inbox.

Comprehensive Task Tracking

- **Activity Timelines:** Each task features a dedicated history view that translates backend audit logs into a readable timeline, showing exactly when a status changed or who reassigned a task.

- **Hierarchical Visualization:** Parent "Stories" visually aggregate their child "Tasks" and "Bugs," giving users a clear overview of feature completion and derived statuses.

Identity and Session Management

- **Profile Customization:** Users can upload and display custom avatar images for their profiles to personalize the team workspace.

- **Persistent Sessions:** The application utilizes a secure refresh token mechanism to maintain user sessions seamlessly in the background, preventing the need for constant re-logins while keeping access tokens secure.

6 API Documentation

6.1 Authentication and Session Security

The platform implements a highly secure, dual-token authentication strategy to manage user sessions while mitigating common web vulnerabilities such as Cross-Site Scripting (XSS).

- **Credential Security:** User passwords are cryptographically hashed using `bcrypt` (with a salt factor of 10) before being persisted to the PostgreSQL database. Raw passwords are never logged or stored.
- **Dual-Token JWT Architecture:**
  - **Access Tokens:** Short-lived JSON Web Tokens (15-minute expiration) are generated upon login. To prevent XSS payload theft, these tokens are never exposed to the frontend JavaScript client. Instead, they are transmitted strictly via `httpOnly` cookies.
  - **Refresh Tokens:** Long-lived tokens (7-day expiration) are generated alongside access tokens and are explicitly persisted in the database. This allows the backend to securely renew active user sessions in the background while maintaining the authority to instantly revoke access across all devices upon explicit logout.

REST API: Authentication Endpoints

Method	Endpoint	Description
`POST`	`/api/auth/register`	Validates payload, hashes the user password, and creates a new database record.
`POST`	`/api/auth/login`	Authenticates credentials, sets the `httpOnly` access cookie, and issues a refresh token.
`POST`	`/api/auth/refresh`	Validates the provided refresh token against the database and issues a fresh access cookie.
`POST`	`/api/auth/logout`	Destroys the active session by clearing the browser cookie and purging the refresh token from the database.

Table 1: Core Authentication API Routes

6.2 Project Management & RBAC Endpoints

The project endpoints enforce the core Role-Based Access Control (RBAC) matrix. The backend middleware strictly verifies the JWT payload and cross-references the `Project_Group` join table to prevent unauthorized data mutation. Access is denied with a `403 Forbidden` response if the user does not meet the required hierarchy tier.

Method	Endpoint (Prefix: /api/projects)	RBAC & Description
GET	/	[**AUTHENTICATED**] Returns all active projects. Dynamically filters queries so users only receive projects they are assigned to (Global Admins bypass filter).
POST	/	[**GLOBAL ADMIN**] Creates a new project and initializes the creator as the Project Admin.
GET	/:projectId	[**PROJECT VIEWER+**] Retrieves specific project metadata and associated nested boards.
PATCH	/:projectId	[**PROJECT ADMIN+**] Updates project details and generates broadcast notifications for team members.
DELETE	/:projectId	[**PROJECT ADMIN+**] Permanently deletes the project, cascading deletion to all boards and tasks.
POST	/:projectId/assign	[**PROJECT ADMIN+**] Upserts user role assignments (Viewer, Member, Admin) and triggers a direct notification.
GET	/:projectId/members	[**AUTHENTICATED**] Retrieves a list of all project users and their respective roles.
PATCH	/:projectId/archive	[**GLOBAL ADMIN**] Toggles the project status to archived, hiding it from default views.
PATCH	/:projectId/unarchive	[**GLOBAL ADMIN**] Restores an archived project to active status.
PATCH	/:projectId/workflow	[**PROJECT ADMIN+**] Updates the JSON definition of the project's custom Kanban workflow and allowed transitions.

Table 2: Comprehensive Project & Authorization Routes

6.3 Board Management Endpoints

The Board endpoints manage the creation and retrieval of Kanban workspaces. A critical performance optimization is implemented in the `GET` route, which utilizes Prisma's nested relational queries to fetch the Board, its Columns, and all associated Tasks in a single, strictly ordered database transaction.

Method	Endpoint (Prefix: /api/boards)	RBAC & Description
`GET`	`/:boardId`	[**PROJECT VIEWER+**] Retrieves the complete board state. Eagerly loads all nested columns and tasks, applying an `orderBy` constraint to guarantee correct visual rendering (left-to-right, top-to-bottom).
`POST`	`/project/:projectId`	[**PROJECT ADMIN+**] Initializes a new board within the specified parent project.
`PUT`	`/:boardId`	[**PROJECT ADMIN+**] Updates the board's display name.
`DELETE`	`/:boardId`	[**PROJECT ADMIN+**] Executes a cascade deletion of the board, wiping all child columns, tasks, and historical audit logs from the database.

Table 3: Kanban Board API Routes

6.4 Task and Issue Management Endpoints

The Task endpoints manage the core lifecycle of work items (Stories, Tasks, and Bugs). These routes implement sophisticated business logic, including automated parent-child status synchronization and a rigid state-machine to enforce valid workflow transitions.

Method	Endpoint (Prefix: /api/tasks)	RBAC & Description
GET	/:taskId	[**PROJECT VIEWER+**] Retrieves full task metadata, including nested comments, assignee/reporter details, and a complete audit trail.
POST	/column/:colId	[**PROJECT MEMBER+**] Creates a new task. The backend automatically calculates the next `order` index and verifies the column's `wipLimit` before persisting.
PATCH	/:taskId	[**CONTEXTUAL**] Updates task attributes. Admins can edit all fields; Assignees or Members (for unassigned tasks) are restricted to status/column transitions.
PATCH	/reorder	[**PROJECT MEMBER+**] Handles bulk Kanban movements. Processes an array of updates in a single `Prisma.transaction` to ensure atomicity during drag-and-drop operations.
DELETE	/:taskId	[**PROJECT MEMBER+**] Permanently removes an issue. Triggers a `syncStoryStatus` check if the deleted item was a subtask of a Story.
POST	/:taskId/comments	[**PROJECT VIEWER+**] Appends a discussion comment to the task, supporting rich-text and user mentions.

Table 4: Task Management and Workflow Endpoints

Automated Hierarchical Synchronization

A significant design feature of the Task controller is the `syncStoryStatus` utility. Whenever a subtask is created, updated, or deleted, the system executes an automated logic gate to derive the parent Story's state:

- **Resolution:** If all subtasks are marked `DONE`, the Story is automatically transitioned to `DONE`.
- **Initialization:** If all subtasks remain in `TO_DO`, the Story remains in `TO_DO`.
- **Progression:** If subtasks exist in mixed states, the Story is automatically promoted to `IN_PROGRESS`, ensuring the Kanban board accurately reflects active development cycles without manual intervention.

6.5 System Tracking & Notification Endpoints

To maintain a rigorous Agile audit trail and keep team members informed of critical workflow updates, the platform implements an automated event-tracking architecture.

A key security feature of this system is that **Audit Logs are strictly read-only**. There are no `POST`, `PUT`, or `DELETE` endpoints exposed for audit logs; they are exclusively system-generated ("Ghost Audits") within secure Prisma transactions whenever a task is modified, ensuring absolute traceability and preventing malicious history tampering.

Method	Endpoint (Prefix: /api/notifications)	RBAC & Description
GET	/	[**AUTHENTICATED**] Retrieves the authenticated user's complete notification feed, sorted chronologically (newest first).
PATCH	/read-all	[**AUTHENTICATED**] A bulk-update utility endpoint that utilizes `updateMany` to instantly mark all of the user's pending notifications as read in a single database query.
PATCH	/:notificationId/read	[**AUTHENTICATED**] Verifies user ownership and marks a specific notification as read, allowing the UI to decrement the unread bell counter.

Table 5: User Notification & Inbox Routes

Implicit Audit Log Retrieval

Rather than exposing a standalone endpoint for audit logs, the historical timeline of an issue is deeply integrated into the Task fetching logic. When a user queries `GET /api/tasks/:taskId`, the backend eagerly loads the nested `auditLogs` array, sorted by `timestamp: 'desc'`. This allows the frontend React application to immediately render a chronological "Activity History" timeline alongside the task details, directly associating each state transition with the `User` who triggered it.

6.6 Kanban Column Configuration Endpoints

The Column endpoints allow Project Admins to fully customize their board's workflow. To bridge the gap between user-defined column names (e.g., "QA Testing") and backend state management, the system strictly maps every dynamic column to a core internal state enum (`cStatus`).

Method	Endpoint (Prefix: /api/columns)	RBAC & Description
POST	/board/:boardId	[**PROJECT ADMIN+**] Creates a new workflow column. The backend automatically calculates its visual `order` index to append it to the right-most edge of the board.
PATCH	/:colId	[**PROJECT ADMIN+**] Partially updates column attributes, allowing Admins to rename phases or enforce/modify Work-In-Progress (`wipLimit`) constraints.
PATCH	/reorder	[**PROJECT ADMIN+**] Handles horizontal drag-and-drop column movements. Takes an array of new order indices and applies them concurrently within a strict `Prisma.$transaction`.
DELETE	/:colId	[**PROJECT ADMIN+**] Deletes a column. **Safety Constraint:** The backend explicitly counts nested tasks and rejects the request (`400`) if the column is not empty, preventing orphaned tasks.

Table 6: Workflow Customization and Column Routes

6.7 User Identity and Profile Endpoints

The User endpoints handle personal profile customization and directory queries. A key feature is the custom Base64 image decoding logic, which natively parses, extracts, and writes uploaded avatar images directly to the server's local filesystem, eliminating the need for external storage dependencies.

Method	Endpoint (Prefix: /api/users)	RBAC & Description
GET	/profile	[**AUTHENTICATED**] Retrieves the authenticated user's profile metadata, including their Global Admin status and avatar link.
PATCH	/profile	[**AUTHENTICATED**] Updates the user's name, email, or profile picture. Base64 image payloads are intercepted, written to the /uploads/avatars directory, and the resulting file path is saved to the database.
PATCH	/password	[**AUTHENTICATED**] Allows users to securely change their password. Requires the current password for verification against the bcrypt hash before accepting the new one.
GET	/	[**CONTEXTUAL**] A highly dynamic endpoint for querying the user directory based on URL parameters: • ?projectId=...: Returns only users assigned to that specific project (Requires Viewer status). • ?notInProject=...: Returns users *not* currently in the project, used for populating the "Add Member" dropdown (Requires Admin status). • **Default:** Returns the entire system directory (Requires Global Admin status).

Table 7: User Profile and Directory Routes

6.8 Task Comments & Collaboration Endpoints

The Collaboration endpoints power the discussion threads within individual tasks. To verify permissions, the backend utilizes a recursive hierarchy-climbing algorithm to trace any subtask, bug, or story back to its root projectId. Furthermore, all comment mutations are securely wrapped in Prisma transactions to ensure the AuditLog is perfectly synchronized with the discussion history.

Method	Endpoint (Prefix: /api/comments)	RBAC & Description
POST	/task/:taskId	[**PROJECT MEMBER+**] Appends a new comment to a task thread. The backend utilizes a regular expression parser (/@\[([^\]@]+)\]\(([^)]+)\)/g) to detect user mentions in the markdown payload, dispatching targeted notifications to tagged users alongside standard broadcast alerts.
PUT	/:commentId	[**AUTHOR ONLY**] Updates the text of an existing comment. Strict ownership validation ensures that only the original author can modify their text. The original content and the edited content are both recorded in the audit trail.
DELETE	/:commentId	[**AUTHOR OR PROJECT ADMIN**] Permanently removes a comment. While authors can delete their own remarks, Project Admins are granted overarching moderation privileges to delete any comment in their workspace. The deleted text is preserved in the read-only AuditLog for accountability.

Table 8: Rich-Text Comments and Mentions API

6.9 Story Management Endpoints

Stories act as top-level Agile containers for standard tasks and bugs. These endpoints are nested under a specific project to automatically link the story directly to the project's backlog.

Method	Endpoint (Prefix: /api/projects/:projectId/stories)	RBAC & Description
`POST`	`/`	[**PROJECT MEMBER+**] Creates a new Story linked directly to the project backlog. Automatically enforces the `STORY` type, sets the initial status to `TO_DO`, and anchors it to the `projectId`.
`GET`	`/`	[**PROJECT VIEWER+**] Retrieves all stories for a specific project, ordered by newest first. Eagerly loads all nested child subtasks (including their current column and board data) to allow the frontend to calculate overall story progress.
`DELETE`	`/:storyId`	[**PROJECT MEMBER+**] Deletes a Story. Executes a secure transaction to safely unlink all child subtasks (setting `parentId` to `null`), converting them into standalone Kanban tasks to prevent accidental data loss.

Table 9: Story Initialization and Tracking Routes

Key Technical Highlights

- **Backlog Initialization:** Stories are created with an `order` of `0` and are directly linked to the `projectId` rather than a specific Kanban column. This accurately mimics an Agile "backlog" item before it is pulled into an active board sprint.
- **Deep Eager Loading:** The `GET` route deeply populates `subtasks.column.board`. This is a highly optimized database query that supplies the frontend with the exact relational data required to render visual progress indicators (e.g., "3 out of 5 subtasks are DONE") in a single network request.
- **Safe Deletion Mechanism:** The `DELETE` route utilizes `Prisma.$transaction` to detach all associated subtasks prior to destroying the parent Story record. This ensures that valuable individual task data is never unintentionally lost when a larger feature epic is cancelled or removed.

7 Backend Testing Architecture

This project features a robust, isolated, production-grade test suite boasting **111 passing tests across 14 suites**. The architecture leverages **Vitest** for extreme execution speed, **Supertest** for in-memory API simulation, and **Vitest-Mock-Extended** for deep-mocking the Prisma database to ensure the real database is never accidentally modified during tests.

7.1 Installation & Tech Stack

To run or contribute to the test suite, ensure the following development dependencies are installed:

```
npm install -D vitest supertest @types/supertest vitest-mock-extended
```

7.2 Running the Tests

The test environment is natively configured in `package.json`. Use the following commands:

- **Run all tests once:** `npm run test`
- **Run tests in watch mode (auto-reloads on file save):** `npm run test:watch`

7.3 Unit Tests (Pure Logic)

Unit tests isolate single functions or pure logic without spinning up the Express server or touching HTTP protocols.

- `workflow.test.ts`: Mathematically verifies the Finite State Machine (FSM) default transitions (e.g., preventing illegal jumps from `DONE` to `TO_DO`).
- `mention.test.ts`: Validates the Regex parser for `@[name](email)` to ensure accurate email extraction for notifications, even with user syntax errors.
- `taskLogic.test.ts`: Tests the `syncStoryStatus` helper to verify that a parent Story correctly calculates its status based on its child subtasks.

7.4 Integration Tests (API + Middleware + Controllers)

Integration tests use Supertest to send virtual HTTP requests through the auth middleware, RBAC controllers, and into the mocked Prisma database.

- `task.test.ts` **(21 tests)**: Verifies task creation, strict Column WIP limits, drag-and-drop reordering array logic, and combined Audit Log/Notification `$transaction` generations.
- `project.test.ts` **(17 tests)**: Checks Global Admin creation rights, RBAC role assignments, and archiving workflows.
- `board.test.ts` **(12 tests)**: Ensures project-board linking and validates `403 Forbidden` responses for unauthorized access attempts.
- `story.test.ts` **(10 tests)**: Tests deep-fetching of nested subtasks and verifies that deleting a story safely orphans (detaches) its subtasks rather than destroying them.
- `user.test.ts` **(10 tests)**: Validates profile updates, Base64 image interception, native file-system avatar creation, and bcrypt password hashing.
- `comment.test.ts` **(9 tests)**: Verifies mention parsing, `USER_MENTIONED` notification dispatching, and strict deletion access control for authors/admins.
- `auth.test.ts` **(9 tests)**: Simulates the JWT lifecycle, including registration, `httpOnly` cookie injection, refresh token renewal, and secure session destruction.
- `column.test.ts` **(8 tests)**: Validates column creation, bulk array updates for reordering, and deletion-blocking for columns containing active tasks.
- `notification.test.ts` **(5 tests)**: Ensures data isolation so users only fetch their own notifications and tests the bulk `read-all` endpoint.
- `error.test.ts` **(2 tests)**: Triggers deliberate database failures (e.g., Prisma `P2025` Record Not Found) to verify the global error handler returns clean `404`s instead of crashing the server.
- `security.test.ts` **(2 tests)**: Probes the `checkAccess` utility to mathematically enforce the Admin > Member > Viewer hierarchy across endpoints.

8 Database Architecture and Data Integrity

To satisfy the rigorous data integrity requirements of the assignment, the application utilizes a relational database schema managed via the Prisma Object-Relational Mapper (ORM). This guarantees type safety from the backend controllers all the way down to the database queries.

8.1 Core Entity Relationships

The database is structured to support complex Agile workflows through the following relational mappings:

- **Many-to-Many RBAC Mapping:** The relationship between `User` and `Project` is handled via an explicit join table named `Project_Group`. This table not only connects users to projects but stores an enum (`PROJECT_ADMIN`, `PROJECT_MEMBER`, `PROJECT_VIEWER`) to instantly resolve a user's permission level for that specific workspace.
- **Hierarchical Issue Tracking:** The `Task` model implements a **self-relation** via a `parentId` foreign key. This elegant design allows standard Tasks and Bugs to be nested underneath top-level Stories. It powers the `syncStoryStatus` controller logic, enabling the parent Story to dynamically calculate its state based on the aggregate status of its children.
- **Cascading Workspaces:** The hierarchy strictly flows as `Project` → `Board` → `Column` → `Task`.

8.2 Data Safety & Concurrency

To prevent data corruption during complex operations (such as reordering Kanban columns or deleting deeply nested issues), the system extensively utilizes `Prisma.$transaction`. This ensures atomicity—if a database write fails halfway through an update, the entire operation rolls back. Additionally, application-level safety locks are implemented, such as explicitly counting nested tasks (`_count.tasks > 0`) and rejecting column deletion requests if the column is not empty, effectively preventing orphaned data.

9 Security Architecture

The application implements a defense-in-depth strategy, securing both the data at rest and data in transit.

9.1 Authentication & Dual-Token JWT Strategy

The platform utilizes a highly secure rotating JSON Web Token (JWT) architecture rather than a monolithic session cookie:

- **Access Token:** A short-lived token generated upon login, transmitted strictly via the HTTP header for authenticating standard API requests.
- **Refresh Token:** A long-lived token stored exclusively in a **Strict, HTTP-only Cookie**. By keeping the refresh token out of the reach of frontend JavaScript, the application is highly resilient against Cross-Site Scripting (XSS) attacks.

When an Access Token expires, the frontend transparently hits the `/api/auth/refresh` endpoint. The backend validates the HTTP-only cookie against the database, issues a fresh Access Token, and rotates the Refresh Token, seamlessly maintaining the user's session.

9.2 Data Encryption & Authorization Middleware

- **Password Hashing:** All user passwords are cryptographically hashed using `bcrypt` with a salt factor of 10 before persisting to the database. Raw passwords are never logged or stored.
- **Route Protection:** Every protected API route passes through an `authenticate` middleware that verifies the JWT payload. Subsequently, a `checkAccess` utility runs a sub-query to verify the user's specific role in the target `projectId`, blocking unauthorized mutation attempts with a `403 Forbidden` response.

10 Functional Requirements Compliance Matrix

The following matrix demonstrates how the final system architecture explicitly satisfies the core requirements outlined in the COP290 evaluation rubric:

Requirement	Implementation Detail	Status
Strict TypeScript	Compiler configured with `strict: true`. Zero usage of the `any` type across the entire codebase.	**Implemented**
Drag and Drop	Implemented using the native HTML5 Drag and Drop API; no third-party libraries were used.	**Implemented**
WIP Limits	Backend controllers intercept column creation/updates and reject additions (`400`) if the count exceeds the limit.	**Implemented**
Hierarchical Tasks	Stories derive their status automatically from child Tasks/Bugs via the custom `syncStoryStatus` algorithm.	**Implemented**
Audit Trail	Immutable, system-generated ghost audits recorded via Prisma transactions on every task mutation.	**Implemented**
Notifications	Persistent in the database. Features bulk-read utilities and targeted parsing for `@mentions` in comments.	**Implemented**

Table 10: Rubric Compliance Overview

11 Setup

Comprehensive documentation (README) for local deployment is provided within the repository.

- **README:** The root `README.md` contains explicit, step-by-step instructions for installing dependencies, configuring the `.env` variables, running Prisma migrations, and spinning up both the Vite development server and the Express backend.
- **API Documentation:** RESTful endpoints are documented above with their expected request bodies, parameters, and authentication requirements.
- **Code Comments:** Complex logic blocks, specifically within the custom Rich Text parser and the task hierarchy controllers, are heavily commented to explain the underlying algorithms and design choices.
- **BACKEND UNIT TESTING DETAILS ARE MENTIONED IN THE README.md FILE**

12 Frontend Architecture & UI/UX

The frontend is a highly interactive, Single Page Application (SPA) designed with a premium, dark-mode-first SaaS aesthetic. Built for speed and productivity, it utilizes strict TypeScript and

isolated CSS Modules to deliver a seamless, Jira-like project management experience without heavy UI libraries.

12.1 Tech Stack

- **Framework:** React 18 + TypeScript + Vite (Ultra-fast HMR)
- **Styling:** Pure CSS Modules (Zero-dependency, locally scoped, custom CSS variables for themeing)
- **Routing:** React Router DOM

12.2 Key Features & Capabilities

- **Interactive Kanban Workspace:**
  - Native HTML5 Drag-and-Drop API implementation for moving tasks across custom columns.
  - Real-time UI updates and visual enforcement of Column WIP (Work In Progress) limits.
- **Dynamic Workflow Builder:**
  - A dedicated UI for Project Admins to visually configure the Finite State Machine (FSM).
  - Uses an interactive matrix to define exact transition rules (e.g., Tasks in "To Do" can only move to "In Progress").
- **Advanced Task & Story Management:**
  - Multi-level hierarchy supporting Parent Stories and Child Subtasks/Bugs.
  - Custom built **Rich Text Editor** featuring floating, Notion-style `@` user-mention dropdowns.
  - Dynamic, color-coded status badges and priority tags (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`).
- **Modern Design System:**
  - Fully custom UI using the `Inter` font family.
  - Context-aware floating modals with `z-index` layering to ensure smooth, uninterrupted user workflows.
  - Global Dark/Light mode theme toggle that seamlessly updates CSS variables across the entire application.
- **Real-time Collaboration UI:**
  - Custom Notification Bell dropdown to view mentions and updates.
  - Role-Based UI rendering (e.g., "Delete" buttons and "Customize Workflow" options are automatically hidden from Viewers/Members and only visible to Project Admins).

13 Code Quality and Testing

For code quality and maintainability, the following practices were strictly implemented:

- **Strict TypeScript Configuration:** The `tsconfig.json` is configured with `strict: true`. We rigorously avoided the use of the `any` type across both the frontend and backend, defining explicit interfaces and types (e.g., `auth.ts`, `kanban.ts`) for all data payloads and state representations.
- **Style Guide Compliance:** The project enforces the Google TypeScript Style Guide. ESLint and Prettier are configured in both the frontend and backend directories to ensure consistent formatting, naming conventions, and clean code structure across all team commits.

- **Backend Unit Testing:** Comprehensive unit tests have been written for the Express API using standard testing libraries. These tests cover critical REST endpoints, business logic, and role-based access control (RBAC) validation to ensure the backend is resilient against unauthorized access and invalid data states.

- **Centralized Error Handling:** The backend utilizes a custom `async_catcher` utility to wrap all asynchronous controller functions. This ensures that any unhandled promise rejections are elegantly caught and forwarded to a centralized error-handling middleware, preventing server crashes and returning standardized HTTP error responses.

14 Conclusion

The Task Board project successfully demonstrates a secure, type-safe, and scalable architecture for agile project management. By integrating a custom Role-Based Access Control (RBAC) system with automated lifecycle tracking and real-time notifications, the platform provides a robust environment for team collaboration. The use of Prisma ORM and strict TypeScript ensured data integrity across the entire stack, fulfilling all assignment requirements.

END OF DOCUMENT